QATIP AWS Lab3

VPC deployment using Terraform (VSC)

Remote Windows VM with VS Code and PowerShell

Contents

Lab objectives 1

Target configuration 2

Before you begin 2

Solution checkpoints 2

Task 1. Create the basic VPC 3

Task 2. Provision the gateways 4

Task 3. Provision routes and security groups 5

Task 4. Create the EC2 instances 7

Task 5. Test connectivity from Windows 10

Task 6. Clean up the lab 12

Troubleshooting 13

References 13

In this lab you build an AWS VPC progressively, add public and private subnets, configure internet and NAT gateways, then deploy two Linux EC2 instances. You test connectivity and change a security group rule to observe its effect before removing the resources.

All editing, Terraform commands and the initial SSH connection run inside the supplied remote Windows VM. After SSH connects to PubVM, commands in that terminal run on the Linux instance. Your personal computer is used only to access the remote Windows desktop.

Lab objectives

- Build a VPC and two subnets in us-west-2.
- Create internet and NAT gateways and the required routes.
- Attach security groups to instances in the public and private subnets.

- Test and deliberately interrupt ICMP connectivity between the instances.
- Remove the Terraform resources and separately clean up the lab key pair.

Target configuration

- **VPC** Test-VPC with 10.1.0.0/16
- **Public subnet** Public-Subnet with 10.1.1.0/24 in us-west-2a
- **Private subnet** Private-Subnet with 10.1.2.0/24 in us-west-2a
- **Public default route** 0.0.0.0/0 to Lab-IGW
- **Private default route** 0.0.0.0/0 to Lab-NAT-GW in the public subnet
- **PubVM** Public IPv4 address and External-SG
- **PrivVM** Private IPv4 address only and Internal-SG

Security groups apply to instance network interfaces, not to entire subnets. The internal group permits traffic from the public subnet CIDR; the external group permits traffic from the private subnet CIDR plus SSH training access. Both groups allow outbound traffic.

Teaching focus: resource references establish most dependencies automatically. An explicit dependency is useful where the relationship cannot be inferred from an argument. Terraform state tracks the resources created and must remain available through cleanup.

Before you begin

1. Complete Lab 0.
2. In VS Code open **C:\awslabs\03**
3. Create an empty **main.tf** in the working folder and save it.

Solution checkpoints

Each task can be attempted independently before reading its example code. Cumulative checkpoints are supplied in **C:\awslabs\solutions\03**: **Task1.tf**, **Task2.tf**, **Task3.tf**, **Task4.tf**, and the final **main.tf**.

To use a checkpoint, destroy any currently deployed resources and copy the contents of the selected file into your working **C:\awslabs\03\main.tf**

Do not run Terraform inside the solutions folder or copy all checkpoint files into the working folder; they contain repeated definitions. Keep the deployment between tasks: each task adds to the previous one. Destruction takes place in Task 6.

Provider compatibility: this revision retains the lab's AWS provider version 4.53.0. In particular, the Elastic IP example uses **vpc = true**, which belongs to that version. Do not mix these examples with snippets for a different provider version.

Task 1. Create the basic VPC

Create the provider configuration, VPC and subnets below in **main.tf**. Use CIDR notation with forward slashes. Keep the **ignore_tags** block when using the QA platform's automatically applied **ca-** tags; it is unnecessary in an account without those tags.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "4.53.0"
    }
  }
}

provider "aws" {
  region = "us-west-2"
  ignore_tags {
    key_prefixes = ["ca-"]
  }
}

resource "aws_vpc" "test_vpc" {
  cidr_block = "10.1.0.0/16"
  tags = {
    Name = "Test-VPC"
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id          = aws_vpc.test_vpc.id
  cidr_block       = "10.1.1.0/24"
  availability_zone = "us-west-2a"
  tags = {
    Name = "Public-Subnet"
  }
}

resource "aws_subnet" "private_subnet" {
```

```
  vpc_id          = aws_vpc.test_vpc.id
  cidr_block      = "10.1.2.0/24"
  availability_zone = "us-west-2a"
  tags = {
    Name = "Private-Subnet"
  }
}
```

Save, then run these commands individually in the **Terraform** PowerShell terminal:

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

Review the plan, then enter **yes** to apply it. Expect **3 managed resources**. In the Oregon VPC console, locate Test-VPC and its two subnets by name and ID. The number of other VPCs depends on the lab account; do not expect a Cloud9 VPC or a fixed total. An explicitly supplied availability zone places the subnet deterministically; if omitted, AWS selects an available zone.

Task 2. Provision the gateways

Add these blocks to the end of **main.tf**. The NAT gateway uses the public subnet and an Elastic IP allocation.

```
resource "aws_internet_gateway" "lab_igw" {
  vpc_id = aws_vpc.test_vpc.id
  tags = { Name = "Lab-IGW" }
}

resource "aws_eip" "static_ip" {
  vpc = true
  tags = { Name = "NAT-GW-Static-IP" }
}

resource "aws_nat_gateway" "lab_nat_gw" {
  allocation_id = aws_eip.static_ip.id
  subnet_id     = aws_subnet.public_subnet.id

  tags = { Name = "Lab-NAT-GW" }
  depends_on = [aws_internet_gateway.lab_igw]
}
```

The NAT gateway's references already establish dependencies on the subnet and Elastic IP. The explicit **depends_on** establishes the additional dependency on the attached internet gateway. There is no need to repeat the Elastic IP in that list.

Save and run:

```
terraform fmt
terraform validate
terraform plan
terraform apply
terraform state list
```

Expect **3 additions and 6 managed resources in total**. A NAT gateway may take several minutes to become available. Verify the internet gateway, Elastic IP and NAT gateway in the Oregon console.

You have created the gateways, but have not yet configured either subnet's internet route. Every VPC has a main route table; each subnet uses it unless associated with another route table. The VPC's local route provides communication within the VPC, subject to security controls.

Task 3. Provision routes and security groups

Add the route tables and associations

Add the following blocks to **main.tf**. The public subnet's default route targets the internet gateway. The private subnet's default route targets the NAT gateway.

```
resource "aws_route_table" "public_route_table" {
  vpc_id = aws_vpc.test_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.lab_igw.id
  }
  tags = { Name = "Public-Route-Table" }
}

resource "aws_route_table" "private_route_table" {
  vpc_id = aws_vpc.test_vpc.id
  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.lab_nat_gw.id
  }
  tags = { Name = "Private-Route-Table" }
}
```

```
resource "aws_route_table_association" "pub_to_ig" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_route_table.id
}

resource "aws_route_table_association" "priv_to_nat" {
  subnet_id      = aws_subnet.private_subnet.id
  route_table_id = aws_route_table.private_route_table.id
}
```

Use **nat_gateway_id** for the NAT gateway target. Using **gateway_id** with a NAT ID can cause a persistent difference between configuration and state. References to the route tables, gateways and subnets provide the required dependencies for these resources.

Do not apply yet; add the security groups below first.

Add the instance security groups

Append these groups. The unrestricted inter-subnet rules and SSH source **0.0.0.0/0** are training settings. In a deployed service, allow only required traffic; restrict SSH to an approved source such as the remote Windows VM's public egress address, rather than your personal computer's address.

```
resource "aws_security_group" "internal_sg" {
  name        = "Internal-SG"
  description = "SG for internal instances"
  vpc_id      = aws_vpc.test_vpc.id

  ingress {
    description = "ALL"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["10.1.1.0/24"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = { Name = "Internal-SG" }
}
```

```
resource "aws_security_group" "external_sg" {
 name        = "External-SG"
 description = "SG for front facing instances"
 vpc_id      = aws_vpc.test_vpc.id

 ingress {
  description = "ALL"
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["10.1.2.0/24"]
 }

 ingress {
  description = "SSH"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }

 tags = { Name = "External-SG" }
}
```

Run **terraform fmt**, **terraform validate**, **terraform plan** and **terraform apply** in the **Terraform** terminal. Expect **6 additions and 12 managed resources in total**. Use **terraform state list** and the VPC resource map to verify them. The groups will be attached to the EC2 instances in Task 4. Security groups are stateful; response traffic for an allowed connection is permitted automatically.

Task 4. Create the EC2 instances

Create the regional key pair

1. In the AWS Console on the remote Windows VM, select **Oregon us-west-2** and open **EC2 > Network and Security > Key Pairs**.

2. Create a key pair named **Oregon_lab_keypair**. Select **RSA** and **.pem**, then create it. Record that you created it for this lab.
3. The browser downloads the private key once. Move it from the remote Windows VM's Downloads folder to **C:\awslabs\03\Oregon_lab_keypair.pem** using File Explorer. Confirm the filename is not suffixed with **(1)** or **.txt**.

If your browser is running on your personal computer, the download is on that computer. Use the training platform's approved transfer method to copy it onto the remote Windows VM. Keep the private key on the remote Windows VM; do not upload it to Git or copy it onto either Linux instance.

If the key pair already exists, do not delete it or assume you have its private key. Ask the instructor whether to reuse it with the matching PEM file or create a unique name. If using a different name, update **key_name** in both instance blocks. Do not delete a reused key during cleanup if another lab still needs it.

Select the AMI

In the Oregon EC2 console, find an instructor-approved, available **Amazon Linux x86_64 EBS-backed AMI**. Use the current AMI ID supplied for the training account, or select an approved Amazon Linux image in the launch wizard and record its ID without launching an instance manually. The SSH username for Amazon Linux is **ec2-user**.

Each instance block below contains an AMI placeholder. Replace both with the same approved Oregon AMI ID. AMI IDs are regional; do not reuse an ID from us-east-1 or us-east-2. If an older hard-coded AMI is no longer available, obtain a replacement before applying.

Confirm placement and access

Both instances inherit the availability zone from their respective subnets. The examples use **t3.micro** with **Standard** CPU credits. Confirm that this type and **us-west-2a** are allowed in the training account. If another Oregon zone is mandated, update both subnet definitions before deployment; changing a subnet's zone later requires replacement.

PubVM receives a public IPv4 address and External-SG. PrivVM receives no public address and uses Internal-SG. Your remote Windows VM needs outbound TCP 22 access to PubVM's public address. Its own management network and remote desktop connection are outside this lab's Terraform state.

Add and deploy the instances

Append the following code to **main.tf**, replacing the AMI placeholder before running Terraform:

```
resource "aws_instance" "private_vm" {
  ami           = "ami-REPLACE_WITH_APPROVED_US_WEST_2_AMI"
  instance_type = "t3.micro"
  key_name      = "Oregon_lab_keypair"
  subnet_id                   = aws_subnet.private_subnet.id
  associate_public_ip_address = false
  vpc_security_group_ids      = [aws_security_group.internal_sg.id]
  credit_specification { cpu_credits = "standard" }
  tags = { Name = "PrivVM" }
}

resource "aws_instance" "public_vm" {
  ami           = "ami-REPLACE_WITH_APPROVED_US_WEST_2_AMI"
  instance_type = "t3.micro"
  key_name      = "Oregon_lab_keypair"
  subnet_id                   = aws_subnet.public_subnet.id
  associate_public_ip_address = true
  vpc_security_group_ids      = [aws_security_group.external_sg.id]
  credit_specification { cpu_credits = "standard" }
  tags = { Name = "PubVM" }
}
```

Also append these outputs, which let you retrieve the two addresses from PowerShell in Task 5:

```
output "pub_vm_public_ip" {
  value = aws_instance.public_vm.public_ip
}

output "priv_vm_private_ip" {
  value = aws_instance.private_vm.private_ip
}
```

Save and run **terraform fmt**, **terraform validate**, **terraform plan** and **terraform apply**. Expect **2 additions and 14 managed resources in total**. Wait for both instances to be running and to pass EC2 status checks. Confirm the security groups and IP assignments in the console. Keep the deployment for testing.

Task 5. Test connectivity from Windows

Prepare the key permissions on the remote Windows VM

1. Keep the **Terraform** terminal open. In VS Code choose **Terminal > New Terminal**.
2. In the new terminal run:

```
Set-Location C:\awslabs\03
Test-Path -LiteralPath .\Oregon_lab_keypair.pem
```

3. Review the supplied **Set-LabKeyPermissions.ps1**. It grants read access to the current Windows user and removes other access entries from this PEM file. Run it as the same user who will run SSH:

```
Unblock-File -LiteralPath .\Set-LabKeyPermissions.ps1
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\Set-LabKeyPermissions.ps1
```

   If organisation policy prevents script execution, use File Explorer: open the PEM file's **Properties > Security > Advanced**, disable inheritance and remove other permission entries so only your current Windows user has read access. Use the instructor's help if the file is owned by another user. Do not change permissions on the entire lab folder.

Connect to PubVM

4. Retrieve the addresses from the current Terraform state:

```
$publicIp = terraform output -raw pub_vm_public_ip
$privateIp = terraform output -raw priv_vm_private_ip
Write-Host "PubVM public IPv4: $publicIp"
Write-Host "PrivVM private IPv4: $privateIp"
ssh -i .\Oregon_lab_keypair.pem "ec2-user@$publicIp"
```

   Record PrivVM's actual private IP before connecting. If the commands fail, check that you applied the outputs from Task 4. On the first SSH connection, verify the host identity with the instructor's information or EC2 console output, then accept the host key if it matches.

   Checkpoint: the prompt changes from Windows PowerShell, such as **PS C:\awslabs\03>**, to an Amazon Linux shell, such as **[ec2-user@ip-10-1-1-10 ~]$**. The SSH terminal now runs commands on PubVM. The **Terraform** terminal remains PowerShell on the remote Windows VM.

Test the security group rule

5. In the **SSH** terminal on PubVM, ping PrivVM's actual private IP. The following is an example; replace **10.1.2.10** with the address recorded in the previous step:

   ```
   ping -c 3 10.1.2.10
   ```

   Expect replies. **-c 3** is Linux ping syntax because this command runs on PubVM. Do not run it in PowerShell on Windows. Windows uses **ping -n 3**, but the remote Windows VM is not the source of this inter-subnet test.

   The VPC's local route connects the two subnets, and Internal-SG permits ICMP from PubVM's subnet. This test does not use the NAT gateway or prove internet access from PrivVM.

6. Keep the SSH session open. In VS Code edit only the ingress **cidr_blocks** in **aws_security_group.internal_sg**, changing **10.1.1.0/24** to **10.1.3.0/24**. Do not change either subnet's CIDR or the rule in External-SG.

7. Save **main.tf**, switch to the **Terraform** PowerShell terminal and run:

   ```
   terraform plan
   terraform apply
   ```

Review the change and enter **yes**. Do not run Terraform inside the SSH shell. Keep the local state in **C:\awslabs\03**.

8. Return to **SSH** and start a new **ping -c 3** command using PrivVM's actual address. Expect no replies because the source no longer matches the allowed CIDR. Allow a short interval after AWS applies the rule; a tracked flow or propagation delay can briefly retain previous behaviour. Stop a lingering ping with **Ctrl+C**.

9. Restore the ingress CIDR to **10.1.1.0/24** in **main.tf**. Save, then run **terraform plan** and **terraform apply** from the **Terraform** terminal.

10. Start another three-packet ping in **SSH**. Replies should return. Finish by running:

    ```
    exit
    ```

The prompt returns to Windows PowerShell. You can now close the SSH terminal. If your remote desktop session disconnects, reconnect to the remote Windows VM; do not assume AWS resources have stopped or been removed.

Task 6. Clean up the lab

1. Confirm that the SSH session has ended. In the **Terraform** PowerShell terminal on the remote Windows VM run:

   ```
   Set-Location C:\awslabs\03
   aws sts get-caller-identity --region us-west-2
   terraform workspace show
   terraform state list
   terraform plan -destroy
   ```

Check the lab account, active workspace and proposed resources. The final checkpoint manages 14 resources in its state. Stop if the plan targets the remote Windows management VM or any unrelated resources. The key pair was created in the console and is not managed by this Terraform configuration.

2. Destroy the resources and enter **yes** after checking the plan:

   ```
   terraform destroy
   ```

Wait for completion. NAT gateway and network deletion can take several minutes. A remote desktop disconnect or an interrupted command does not confirm cleanup. Keep the configuration, state and credentials available until destruction succeeds; resolve any reported failure and retry from the same folder and workspace.

3. Verify the state and the console:

   ```
   terraform state list
   ```

   The command should complete successfully with no managed resource addresses. Confirm that PubVM and PrivVM are terminated, Test-VPC is gone, and the lab NAT gateway has been deleted and its Elastic IP released in us-west-2. Deleted NAT gateways and terminated instances can remain visible in the console for a while; check their status. Do not remove the remote Windows VM or its management network.

4. If you created **Oregon_lab_keypair** specifically for this lab and no other instances use it, delete it in **EC2 > Key Pairs** in us-west-2. Alternatively, with the correct lab credentials:

   ```
   aws ec2 delete-key-pair --key-name Oregon_lab_keypair --region us-west-2
   ```

   Use the actual name if you chose a unique one. Keep a reused key pair if another exercise requires it. Deleting an EC2 key pair does not itself remove its downloaded PEM file or revoke access on already running instances.

5. After confirming destruction and that the key is no longer needed, remove the private key from the remote Windows VM:

   Remove-Item -LiteralPath .\Oregon_lab_keypair.pem

   Also remove any duplicate PEM download from that VM's Downloads folder and any transfer copy on your personal computer. Do not delete the Terraform state as a substitute for destroying resources. You have completed the lab when both Terraform-managed resources and any lab-only key material have been cleaned up.

Troubleshooting

SSH times out

Check PubVM's running status, public IP, External-SG inbound TCP 22 rule, public route to the attached internet gateway and outbound TCP 22 from the remote Windows VM. If SSH has been restricted to a /32 source, use the egress address of that VM rather than your personal device. PrivVM has no public IP and is not the initial SSH target.

SSH rejects the key

Check the exact PEM path and the key pair name on PubVM. Use **ec2-user** for the selected Amazon Linux AMI. Restrict the file's permissions as described in Task 5. A key downloaded by another Windows user may require an ownership correction. A permission error differs from a network timeout.

The baseline ping fails

Check PrivVM's actual private IP, the two subnet CIDRs, VPC placement, associated security groups and Internal-SG's **10.1.1.0/24** ingress rule. If your independent build changed network ACLs or an instance firewall, investigate those too. The initial deployment leaves the VPC's default network ACL unchanged. Ensure you run Linux ping on PubVM and apply Terraform changes from the Windows terminal.

Terraform reports missing variables or duplicate resources

Check the working folder. Do not place several cumulative checkpoint **.tf** files in one working directory. Terraform reads them together, not as alternative versions.

References

- AWS provider 4.53.0: https://registry.terraform.io/providers/hashicorp/aws/4.53.0/docs

- AWS SSH prerequisites: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connection-prereqs-general.html
- AWS SSH troubleshooting: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/TroubleshootingInstancesConnecting.html
- AWS AMI selection: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/finding-an-ami.html
- Terraform destruction: https://developer.hashicorp.com/terraform/cli/commands/destroy
- Terraform state: https://developer.hashicorp.com/terraform/language/state

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.